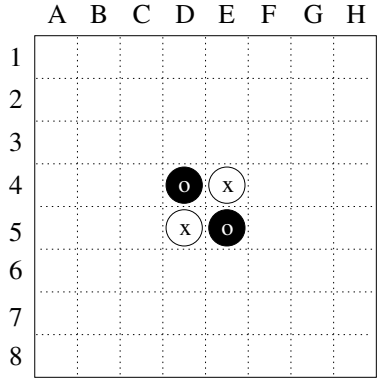
Práctica 2: Algoritmos Minimax y $\alpha - \beta$. Heurísticas para juegos.

1. Planteamiento del problema

El primer objetivo de esta práctica, para desarrollar en dos sesiones, es entender e implementar el algoritmo $\alpha - \beta$ para resolución en árboles de juego. El algoritmo deberá implementarse del modo más general posible, pero se probará aplicándolo al juego del Otello, para el que además de deben realizar varias heurísticas de juego y hacerlas competir unas contra otras, tanto las realizadas por componentes de la misma pareja de prácticas como las de otras parejas.

Las reglas del Otello son: sea un tablero cuadrado de 64 casillas (8 filas y 8 columnas). Existen 64 fichas en forma de moneda con colores o símbolos diferentes en cada cara (como las monedas reales). En nuestro caso, digamos una cara blanca y una negra. La configuración inicial es la que se muestra en la figura.



A cada jugador se le asocia un color (cara) de las fichas. Empezando por el jugador a quien se asigna negro, cada jugador sitúa una ficha en cualquiera de las casillas libres con la condición de que al hacerlo se pueda revertir al menos una ficha del color del adversario. Una ficha se debe revertir si está alineada en horizontal, vertical o diagonal entre dos del color opuesto, y siempre que toda la línea entre éstas sea continua (sin vacíos) y del mismo color. Esto significa p. ej. que desde la posición de inicio, la primera jugada de negra sólo podría ser a E3 (volteando entonces la blanca en E4), a C5 (volteando D5), a F4 (volteando E4) o a D6 (volteando D5).

Si un jugador no puede colocar ninguna ficha de su color de acuerdo a estas reglas, debe pasar, y sólo en ese caso se le permite hacerlo. El juego acaba cuando las 64 fichas están colocadas, o cuando ninguno de los dos jugadores puede colocar ninguna ficha más. Gana el jugador cuyo color se muestre en ese momento en más fichas del tablero, o los dos jugadores empatan en caso de igualdad en número de fichas.

El resultado entregable es:

- La implementación del algoritmo $\alpha - \beta$ y su prueba en el juego.
- La implementación de al menos dos heurísticas y su prueba en el juego jugando una contra la otra.
- Un archivo `explic.txt` en texto ASCII con las instrucciones de compilación y resultados obtenidos con cada heurística.

Se entregará el código realizado SUFICIENTEMENTE COMENTADO.

Apéndice: API de C++

Se definen algunos tipos y se proporciona una clase `Board` para facilitar la implementación.
Los tipos son:

- Tipo Square: `typedef pair<char,char>Square`. Es un par de dos caracteres que representa una casilla del tablero. Para ser válido el primer carácter debe ser '1',...,'8' (la fila), y el segundo, 'A',...,'H' (la columna). Nótese que los números están representados como caracteres, no como enteros.
- Tipo Sqlist: `typedef list<Square>Sqlist`. Una lista de casillas.
- Tipo Token: definido por enumeración como `enum Token {x,o,e }`. La o representa negro, la x blanco y la e vacío (empty).

- Tipo Player: definido por enumeración como `enum Player {Bob,Alice}`. Por convenio, Alice juega con negras (token 'o') y por tanto abre el juego.
- Tipo Move: `typedef pair<Player,Square>Move`. Representa un movimiento, donde la primera componente es el jugador que va a realizarlo y la segunda, la casilla donde pondrá la ficha de su color.

Las funciones auxiliares son:

- `string Name(Player pl)`: Toma un jugador (como tipo Player) y devuelve una string con su nombre. (Pensada para mostrar por pantalla).
- `Player Opponent(Player pl)`: Toma un jugador como tipo Player y devuelve su oponente, también como tipo Player.
- `Token Tk(Player pl)`: Toma un jugador como tipo Player y devuelve el valor (como tipo Token) de la ficha con las que juega.
- `char Symbol(Token tk)`: Toma una ficha (como tipo Token) y devuelve un carácter. (Pensada para imprimir por pantalla).
- `bool ValidSquare(Square sq)`: Toma una casilla y devuelve true si es válida y false si no. Véase arriba las restricciones a los caracteres del par para el tipo Square.
- `bool ValidPlayer(Player pl)`: Toma un jugador y devuelve true si es uno de los dos jugadores válidos y false si no.

La clase se llama `Board`, y su interface público es:

- `Board()`: Crea un tablero en el estado inicial (es decir, con dos fichas 'o' en ('4','D') y ('5','E') y dos fichas 'x' en ('5','D') y (4,'E')).
- `void Show(ostream &outst)`: Muestra por pantalla (pasando cout ó cerr) o en cualquier otra stream de salida el estado actual del tablero.
- `Token Content(Square sq)`: Devuelve el token (x,o,e) que ocupa la casilla que se le pasa.
- `bool IsLegal(Move mv)`: Devuelve `true` si el movimiento pasado puede ejecutarse en el estado actual del tablero y `false` si no.
- `void MakeMove(Move mv)`: Efectúa el movimiento que se le pasa, si es legal (no es necesario comprobar de antemano, esta función lo hace), lo cual altera el estado interno del tablero (del objeto que llama a esta función).
- `Board TryMove(Move mv,bool &legal)`: Si el movimiento que se le pasa es legal (lo comprueba internamente), copia el tablero actual en uno nuevo, ejecuta el movimiento sobre éste, y devuelve dicha copia, con lo que el objeto que llama a esta función no se altera. La variable `legal`, pasada por referencia, vuelve con el valor obvio según el movimiento pueda ejecutarse o no.
- `bool GameOver(Player &winner,unsigned &winnerpoints,unsigned &looserpoints)`: Devuelve `true` si el juego ha terminado, es decir, si ninguno de los contendientes puede realizar más jugadas (por bloqueo, o porque el tablero está lleno) y `false` en caso contrario. Además, si el juego ha terminado, las variables pasadas por referencia vuelven llenas con respectivamente el ganador (como tipo Player), el número de fichas del ganador y el número del perdedor (como tipo `unsigned`).
- `Sqlist ValidMoves(Player pl)`: devuelve una lista de casillas en las que al jugador que se le pasa se le permite poner una ficha (por supuesto, de su color)

La función heurística está definida en los archivos `heuristic.h/cpp`. Se provee la heurística no informativa (devuelve siempre 0) y el ejercicio es hacer otra u otras mejores, usando las funciones que se considere necesario de la clase `Board` (esencialmente, `Content` para ver qué fichas hay en el tablero y dónde están). Deberían programarse al menos dos heurísticas, y hacerlas jugar una contra otra. Para ello la función heurística puede implementarse en la forma

```
#include "heuristic.h"

float h(Player pl,Board b)
{
 if (pl==Bob)
 {
  // Program one heuristic here...
  // return ...
 }
 else
 {
  // Program a different one here...
  // return ...
 }
}
```

Además, el programa principal juega con un algoritmo minimax, definido en `Minimax.h/cpp`. La implementación está detalladamente comentada. El ejercicio es copiar estos dos archivos en otros, `Alfabeta.h/cpp` y modificar todo lo necesario para escribir un $\alpha-\beta$, probándolo luego en el programa principal en sustitución del minimax.